👩‍🏫 RAMÓN ABRAMO • CURSO DE DESARROLLO WEB 2026

El Juego del Ahorcado en JavaScript

Guía pedagógica integral, especificación de hitos de desarrollo y desglose comparativo de tres soluciones técnicas progresivas.

El **Juego del Ahorcado** es uno de los proyectos didácticos más completos en la formación de desarrolladores web frontend. Integra el manejo de arrays y cadenas, manipulación del DOM, captura de eventos de usuario y dibujo vectorial progresivo con SVG en base al estado del juego.

Índice General del Documento

Sección	Contenido Pedagógico	Página
1. Presentación	Contexto docente, objetivos formativos y resumen del ejercicio.	Página 1
2. Enunciado Oficial	Ciclo de vida del juego, reglas, dibujo del muñeco y flujo de datos.	Página 2
3. Los 6 Hitos	Desglose de fases obligatorias de desarrollo y pruebas de validación.	Página 3
4. Gráficos SVG	Lienzo vectorial, coordenadas de la horca y revelado por CSS.	Página 4
5. Tres Soluciones	Matriz comparativa: V1 Básica, V2 Teclado Virtual y V3 POO & Audio.	Página 5
6. Rúbrica & Docencia	Pautas metodológicas, temporización en el aula y rúbrica de evaluación.	Página 6

Competencias Técnicas que Desarrolla el Alumno

Lógica Algorítmica

Gestión de arrays paralelos, búsquedas con bucles `for` e `includes()`, y control de vidas.

Manipulación del DOM

Creación dinámica de elementos, manipulación de clases semánticas y actualización reactiva de la interfaz.

Arquitectura de Eventos

Uso de *Event Bubbling*, sincronización con teclado físico y desacoplamiento mediante clases POO.

1. Enunciado Oficial del Ejercicio

> El alumnado debe implementar la lógica de un juego del ahorcado donde el sistema elige una palabra al azar de un banco predeterminado, oculta sus caracteres mediante guiones bajos y permite al usuario descubrirla proponiendo letras una a una.

Reglas Clave de la Dinámica

1. **Acierto:** Si la letra propuesta está en la palabra secreta, se revelan *todas* sus apariciones en el display.
2. **Fallo:** Si la letra no coincide, se descuenta 1 vida y se dibuja la siguiente parte del muñeco ahorcado.
3. **Validación preventiva:** Proponer una letra repetida o un carácter no alfabético *nunca* debe descontar vidas.
4. **Victoria:** Se produce cuando no queda ningún guion bajo en la palabra adivinada.
5. **Derrota:** Se produce al cometer 6 fallos consecutivos. Se revela la palabra y el muñeco muestra cara de derrota.

2. Diagrama de Flujo del Estado de Partida

Flujo de Ejecución por Jugada

`[Entrada de Letra]` ➔ ¿Es válida y no usada? ➔ SI ➔ Añadir a `letrasUsadas`.
➔ ¿Está en `palabraSecreta`?
  ├─► **SÍ:** Destapar posiciones en `letrasAdivinadas` ➔ ¿Quedan guiones? ➔ NO ➔ 🎉 **VICTORIA**.
  └─► **NO:** Incrementar `fallos++` ➔ Dibujar parte SVG ➔ ¿Fallos == 6? ➔ 💀 **DERROTA**.

3. Estructuras de Datos Fundamentales

Variable / Estructura	Tipo	Propósito Pedagógico
`palabraSecreta`	String	Almacena la palabra seleccionada al azar (ej: `"JAVASCRIPT"`).
`letrasAdivinadas`	Array <String>	Contiene guiones y letras acertadas (ej: `['_', 'A', '_', 'A']`).
`letrasUsadas`	Array o Set	Historial de caracteres ya propuestos para evitar penalizaciones dobles.
`fallos`	Number	Contador de 0 a 6 que determina directamente qué piezas del SVG mostrar.

Los 6 Hitos Obligatorios de Construcción

> El desarrollo incremental garantiza que el alumnado valide visualmente cada funcionalidad antes de continuar con la siguiente:

Hito	Objetivo Técnico	Detalle de Implementación	Prueba de Aula
H1	**Palabra & Display**	Elegir palabra aleatoria con `Math.random()` y crear un array de guiones bajos de igual longitud.	Palabra de 4 letras genera exactamente `_ _ _ _`.
H2	**Validación Letra**	Filtrar entradas vacías, números o caracteres especiales. Comprobar no duplicidad en el historial.	Probar dos veces 'A' no suma fallo ni modifica vidas.
H3	**Descubrir Aciertos**	Recorrer la palabra con un bucle. Si coincide la letra, reemplazar el guion en la misma posición.	'A' en "PANTALLA" desvela las 3 apariciones juntas.
H4	**Dibujo Progresivo**	Si la letra no existe, incrementar `fallos` y activar la clase `.visible` en la parte SVG correspondiente.	Con 1 fallo se ve la cabeza; con 2 el tronco...
H5	**Victoria**	Verificar si `!letrasAdivinadas.includes('_')`. Si es verdad, emitir mensaje de felicitación y deshabilitar inputs.	Adivinar la última letra bloquea el teclado y felicita.
H6	**Derrota & Fin**	Si `fallos ⩾ 6`, mostrar expresión de derrota en el muñeco, destapar la palabra oculta y bloquear inputs.	Al 6º fallo el juego termina y muestra la solución.

Estrategia Didáctica para el Aula

Paso 1: La prueba en consola

Antes de conectar el HTML, el alumno puede imprimir `console.log(letrasAdivinadas.join(' '))` tras cada jugada para entender la sincronía entre memoria y pantalla.

Paso 2: La sincronía visual

Asegurar que las vidas mostradas en texto coincidan estrictamente con las partes dibujadas en el SVG (6 - fallos).

El Lienzo Vectorial SVG del Ahorcado

> El juego utiliza un elemento `<svg viewBox="0 0 200 240">` que garantiza escalabilidad infinita, nitidez en cualquier resolución y un consumo de recursos prácticamente nulo.

1. Estructura de Elementos y Coordenadas

Parte	Etiqueta SVG	Coordenadas / Parámetros	Estado Inicial
Base	`<line>`	`x1="20" y1="220" x2="100" y2="220"`	Siempre visible
Poste & Viga	`<line>`	`x1="50" y1="220" x2="50" y2="20"` (poste) y `x2="140"` (viga)	Siempre visible
Cuerda	`<line>`	`x1="140" y1="20" x2="140" y2="50"`	Siempre visible
1. Cabeza	`<circle>`	`cx="140" cy="70" r="20"`	Oculta (clase `.parte-oculta`)
2. Tronco	`<line>`	`x1="140" y1="90" x2="140" y2="150"`	Oculta
3. Brazo Izquierdo	`<line>`	`x1="140" y1="105" x2="115" y2="135"`	Oculta
4. Brazo Derecho	`<line>`	`x1="140" y1="105" x2="165" y2="135"`	Oculta
5. Pierna Izquierda	`<line>`	`x1="140" y1="150" x2="120" y2="195"`	Oculta
6. Pierna Derecha	`<line>`	`x1="140" y1="150" x2="160" y2="195"`	Oculta

2. El Mecanismo de Revelado con CSS

```css
/* main.css */
.parte-oculta {
  opacity: 0;
  transition: opacity 0.3s ease;
}
.parte-oculta.visible {
  opacity: 1;
}
```

> **Ventaja Didáctica de la Solución Vectorial**
>
> No requiere cargar 7 imágenes PNG separadas ni lidiar con rutas rotas. Todo el dibujo reside en el propio documento HTML y se activa limpiamente mediante `partes[fallos - 1].classList.add('visible')`.

Cuadro Comparativo de las Tres Versiones

> Se proporcionan tres implementaciones completas e independientes para adaptarse al ritmo y perfil de aprendizaje de cada grupo:

Criterio	V1: Básica Lineal	V2: Teclado & Eventos	V3: Avanzada POO
Carpeta	`version-1-basica-lineal/`	`version-2-teclado-eventos/`	`version-3-avanzada-poo/`
Paradigma	Imperativo lineal directo	Basado en eventos y UX	Orientado a Objetos (POO)
Entrada del usuario	`<input type="text">` clásico	Teclado virtual + Teclado físico	Teclado virtual + Físico accesible
Estructura del Banco	Array simple de strings	Array de objetos (palabra, cat, pista)	Banco clasificado con dificultad
Manejo de eventos	Listeners en botones fijos	**Event Bubbling** en el padre	Desacoplado (UI vs Engine)
Estructura de datos	Array básico (`includes` , `push`)	`Set` **nativo** sin duplicados	Estado encapsulado en clase
Persistencia	Ninguna (en memoria)	Ninguna	`localStorage` **(rachas y %)**
Efectos de sonido	Sin audio	Sin audio	**Web Audio API puro nativo**
Efectos visuales	SVG estático con opacidades	Casillas animadas y corazones	**Lluvia de confeti Canvas**
Niveles de dificultad	Fija (6 vidas)	Fija (6 vidas)	**3 Niveles + Contrarreloj 45s**
Recomendación docente	**Iniciación y fundamentos**	**Práctica estándar guiada**	**Ampliación y portafolio**

Progresión Pedagógica Sugerida

Semana 1: V1

Explicar variables, arrays, bucles `for` e `if/else`. Consolidar el flujo imperativo.

Semana 2: V2

Añadir UX profesional: teclado dinámico, delegación de eventos y objetos con pistas.

Semana 3: V3

Subir de nivel: Clases ES6, Web Audio API, Canvas y persistencia con `localStorage`.

Pautas Docentes y Rúbrica de Calificación

> Criterios recomendados para la evaluación objetiva de la práctica entregada por el alumnado:

Criterio de Evaluación	Descripción Técnica	Ponderación
1. Lógica de Acierto y Guiones	El array de guiones refleja la longitud exacta y destapa todas las letras coincidentes.	**25%**
2. Sincronía del Dibujo SVG	Cada fallo activa la pieza corporal correspondiente sin desfasarse del contador de vidas.	**25%**
3. Validación Preventiva	Letras repetidas o caracteres no válidos muestran alerta y no penalizan al jugador.	**20%**
4. Fin de Juego y Bloqueo	Al ganar o perder, los controles se bloquean y se revela el resultado sin bugs.	**15%**
5. Calidad del Código	Nombres de variables semánticos, indentación correcta y ausencia de errores en consola.	**15%**

Errores Típicos del Alumnado y Cómo Anticiparlos

1. El error del "primer acierto único"

Muchos alumnos usan `indexOf()` y solo descubren la primera 'A' de una palabra como "CASA". Recuerda explicar que se debe usar un bucle `for` completo para recorrer todos los índices.

2. Descontar vida en letras repetidas

Si el usuario vuelve a teclear una letra ya probada, el código debe abortar con `return` antes de llamar a la lógica de acierto/fallo.

Material Proporcionado en este Repositorio

- 📁 Código maestro para el profesor (`app.js`)
- 📝 Plantilla guiada con TODOs para los alumnos (`app.alumnos.js`)
- 🔑 Clave canónica de corrección limpia (`app.solucion.js`)
- 🖥️ Presentaciones HTML interactivas con pantalla completa para clase.